On Combiners for Verifiable Random Functions

Basil Rohner (brohner@ethz.ch)
Bachelor's Thesis, ETH Zürich
April 14, 2026

Research Question

Can we *combine* multiple instances of verifiable random function (VRFs) into a single VRF that is secure as long as *enough* of the instances are?

- How do we *combine* the candidate schemes?
- What are *enough* candidates to combine?

Can we combine *k* out of *n* VRF instances in a *black-box* fashion?

Verifiable Functions

Definition (Verifiable Function)

A *verifiable function* (VF) VF is a triple of the following efficient algorithms:

- $\mathsf{Gen}_{\mathsf{VF}}(1^\lambda) = (\mathsf{sk}, \mathsf{pk})$ generates a key-pair
- $\mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x) = (y, \pi)$ evaluates points to an image-proof pair
- $\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = b$ verifies evaluations

A VF is a purely syntactic object, we define security properties on top of it.

Properties of Verifiable Functions

ε-Correctness

Honestly generated image-proof pairs verify with probability at least ε:

$$\Pr\left[\begin{array}{l}(\mathsf{sk},\mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda}) \\ \quad x \leftarrow_{\$} \{0,1\}^{\lambda} \quad : \mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk},x,y,\pi) = 1 \\ (y,\pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk},x)\end{array}\right] \geq \varepsilon(\lambda)$$

VF is *correct* if it is 1-correct and *statistically correct* if it is $(1-\varepsilon)$-correct and $\varepsilon \in \mathsf{negl}(\lambda)$.

Properties of Verifiable Functions

Unique Provability

For every pk and every x, at most one image y is provable:

$$\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y_0, \pi_0) = \mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y_1, \pi_1) = 1 \implies y_0 = y_1.$$

Properties of Verifiable Functions

(ε, t)-Pseudorandom

For any adversary $\mathcal{A}$ running in time at most t, playing the prf experiment $\mathbf{Exp}^{\text{prf}}_{\text{VF}}(\lambda, \mathcal{A})$:
The advantage $\mathbf{Adv}^{\text{prf}}_{\text{VF}}(\lambda, \mathcal{A})$ is bounded from above by

$$\mathbf{Adv}^{\text{prf}}_{\text{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\text{prf}}_{\text{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

Properties of Verifiable Functions

(ε, t)-Unpredictability

For any adversary $\mathcal{A}$ running in time at most t, playing the prf experiment $\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$:
The advantage $\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ is bounded from above by

$$\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

Properties of Verifiable Functions

$(\varepsilon, \mathfrak{t})$-Weak Pseudorandomness

For any adversary $\mathcal{A}$ running in time at most $\mathfrak{t}$, playing the prf experiment $\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$, the advantage $\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ is bounded from above by:

$$\mathbf{Adv}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

A VF is *pseudorandom* if it is $(\varepsilon, \mathfrak{t})$-pseudorandom for $\varepsilon \in \mathsf{negl}(\lambda)$ and $\mathfrak{t} \in \mathsf{poly}(\lambda)$.

Pseudorandomness experiment

Experiment $\mathbf{Exp}^{\mathrm{prf}}_{\mathrm{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathrm{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_{\$} \mathcal{A}_1^{\mathcal{O}_{\mathrm{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathrm{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_{\$} \mathcal{Y}$
7 : $b' \leftarrow_{\$} \mathcal{A}_2(1^{\lambda}, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Oracle $\mathcal{O}_{\mathrm{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathrm{VF}}(\mathsf{sk}, x)$
3 : **return** y

Verifiable Random Functions

Verifiable Random Function

A *verifiable random function* (VRF) is a VF that is:

- Correct
- Uniquely Provable
- Pseudorandom

Robust Black-Box Combiners

Definition (Robust $(n; k)$ Black-Box P-Combiner)

A ppt. oracle Turing machine Π_P that, given oracle access to n candidates $\mathcal{O}_1, \ldots, \mathcal{O}_n$, implements primitive P securely whenever at least k of the candidates do s.t. computational security properties hold against inefficient adversaries.

- **Black-box:** the combiner queries candidates as oracles — it cannot inspect or modify their code.
- We require security against adversaries with access to a **PSPACE oracle**, forcing the combiner to derive its security from the candidates.
- Known results: simple $(n; 1)$ black-box combiners exist for PRFs, OWFs, PKE, key agreement, … — but **not known for VRFs**.

Robust Black-Box DAG Combiners

We consider robust black-box combiners in a DAG normal form.

- Dedicated **source** s (the input) and **sink** t (the output).
- Every other vertex v carries
  - a label $\ell(v) = (i, j)$ specifying *which candidate's* j-th oracle is queried, and
  - a ppt. Turing machine f^v that computes the query input from the responses of the predecessors $\mathcal{N}_{\mathcal{G}}^{-}(v)$.
- Execution traces the DAG in topological order: at each v, build $\mathbf{q}_v \leftarrow f^v(\{\mathbf{x}_w\}_{w \in \mathcal{N}_{\mathcal{G}}^{-}(v)})$ and set $\mathbf{x}_v \leftarrow \mathcal{O}_{\ell(v)}(\mathbf{q}_v)$. The sink's Turing machine f^t produces the final output.

Equivalence. Every robust $(n; k)$ black-box combiner has a *computationally equivalent* DAG combiner — the DAG form is a convenient normal form for our proofs.

; (t) at (0,-10)

I

; [-latex] (x) -> (v1); [-latex] (v1) -> (t); [-latex] (x) -> (v2); [-latex] (v2) -> (t);

Equivalence

- Robust $(n; k)$ black-box DAG combiners are robust $(n; k)$ black-box combiners
- Robust $(n; k)$ black-box combiners can be converted into robust $(n; k)$ black-box DAG combiner

By induction over the number of oracle queries, bounded from above by $p(\lambda) \in \mathrm{poly}$. $p(\lambda)$ layers. At every vertex v in layer l simulate the combiner Π_{VRF}

The 1-out-of-2 Combiner

- Simplest non-trivial case, only two candidates
- Construction extends to robust $(n-1)$-out-of-n black-box combiner
- Impossibility extends to robust $\lfloor n/2 \rfloor$-out-of-n black-box combiner

Proof Setup

Suppose Π_{VRF} is a robust $(2;1)$ black-box DAG VRF-combiner.

- $\mathcal{O}_{vrf}$: a secure VRF
- $\mathcal{O}_{uppc}$: a *trivial*, uniquely provable and perfectly correct VF:
  - $\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) = (\mathsf{sk}_{def}, \mathsf{pk}_{def})$
  - $\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = (y_{def}, \pi_{def})$
  - $\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = 1 \iff y = y_{def}$
- $\mathcal{O}_{vrf}$ a secure VRF $\implies$ $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{\mathsf{vrf}}, \mathcal{O}_{uppc}}$ a secure VRF

Goal: Adversary $\mathcal{A}^{\mathsf{PSPACE}, \mathcal{O}_{\mathsf{vrf}}, \mathcal{O}_{uppc}}$ against psuedorandomness of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{\mathsf{vrf}}, \mathcal{O}_{uppc}}$.
Assume: $\mathcal{SK} = \{0,1\}^{s(\lambda)}$ for $s \in \mathsf{poly}(\lambda)$ for all candidates.

Proof Overview

Assume we have a Turing machine (inefficient) $\mathsf{EvalSim}_{\mathsf{VRF}}$ satisfying:

- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \top$ if $\mathsf{sk}_{cand} = \mathsf{sk}$
- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \bot$ if $\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x) \neq \mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}, x)$
- $\mathsf{EvalSim}_{\mathsf{VRF}}$ makes *no queries to any oracles*.
- $\mathsf{EvalSim}_{\mathsf{VRF}}$ has a *polynomial space complexity*.

For $x \in \{0,1\}^{\lambda}$, $(y, \pi) = \Pi^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}_{\mathsf{VRF}} \mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$ and $\tau = \mathsf{trace}(\Pi^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi))$.

Proof Overview - Adversary

Adversary $\mathcal{A}$

1 : $b \leftarrow_\$ \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_\$ \mathsf{Gen}_{\mathsf{VF}}(1^\lambda)$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_\$ \mathcal{A}_1^{\mathcal{O}_{\mathsf{eval}}}(1^\lambda, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_\$ \mathcal{Y}$
7 : $b' \leftarrow_\$ \mathcal{A}_2(1^\lambda, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Oracle $\mathcal{O}_{\mathsf{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_\$ \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
3 : **return** y

Proof Overview - How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

Given $(\mathsf{sk}_{cand}, x, y, \pi, \tau)$, $\mathsf{EvalSim}_{\mathsf{VRF}}$ is computed by:

- Follow the DAG $\mathcal{G}_{\mathsf{Eval}}$, describing $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$, in some topological ordering
- For every vertex v corresponding to a query to $\mathcal{O}_{uppc}$ answer with default value
- For every vertex v corresponding to a query to $\mathcal{O}_{vrf}$ check τ:
  - If $\mathbf{q} \in \tau$: answer accordingly
  - If $\mathbf{q} \notin \tau$: branch for every possible answer
- Return $\top$ iff all branches produce the image y

Proof Overview - How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

Given $(\mathsf{sk}_{cand}, x, y, \pi, \tau)$, $\mathsf{EvalSim}_{\mathsf{VRF}}$ is computed by:

- Follow the DAG $\mathcal{G}_{\mathsf{Eval}}$, describing $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$, in some topological ordering
- For every vertex v corresponding to a query to $\mathcal{O}_{uppc}$ answer with default value
- For every vertex v corresponding to a query to $\mathcal{O}_{vrf}$ check τ:
  - If $\mathbf{q} \in \tau$: answer accordingly
  - If $\mathbf{q} \notin \tau$: branch for every possible answer
- Return $\top$ iff all branches produce the image y

Proof Overview - How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

Assume we have a Turing machine (inefficient) $\mathsf{EvalSim}_{\mathsf{VRF}}$ satisfying:

- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \top$ if $\mathsf{sk}_{cand} = \mathsf{sk}$
- $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \bot$ if $\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x) \neq \mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}, x)$
- $\mathsf{EvalSim}_{\mathsf{VRF}}$ makes *no queries to any oracles*.
- $\mathsf{EvalSim}_{\mathsf{VRF}}$ has a *polynomial space complexity*.

For $x \in \{0,1\}^{\lambda}, (y, \pi) = \Pi^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}_{\mathsf{VRF}}\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$ and $\tau = \mathsf{trace}(\Pi^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi))$.

Proof Overview - How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

$\mathsf{EvalSim}_{\mathsf{VRF}}$ has a *polynomial space complexity*:

- $\mathsf{EvalSim}_{\mathsf{VRF}}$ in NPSPACE since every branch is an execution of $\mathsf{Eval}_{\mathsf{VRF}}$
- NPSPACE $\subseteq$ PSPACE

Proof Overview - How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

$\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \perp$ if $\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x) \neq y$:

- All queries made to $\mathcal{O}_{uppc}$ answered the same $\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x)$
- All queries made to $\mathcal{O}_{vrf}$ with $\mathbf{q} \in \tau$ answered the same
- For all queries made to $\mathcal{O}_{vrf}$ with $\mathbf{q} \notin \tau$ there exists a branch which is the same

$\Longrightarrow$ At least one branch evaluates to $\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x) \neq y$.

Proof Overview - How to get $\mathsf{EvalSim}_{\mathsf{VRF}}$

$\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x, y, \pi, \tau) = \top$ if $\mathsf{sk}_{cand} = \mathsf{sk}$.

Suppose $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) = \bot$:

Proof Overview

Suppose Π_{VRF} is a robust $(2;1)$ black-box DAG VRF-combiner.

- $\mathcal{O}_{vrf}$: a secure VRF
- $\mathcal{O}_{uppc}$: a *trivial*, uniquely provable and perfectly correct VF:
  - $\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) = (\mathsf{sk}_{def}, \mathsf{pk}_{def})$
  - $\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = (y_{def}, \pi_{def})$
  - $\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = 1 \iff y = y_{def}$
- $\mathcal{O}_{vrf}$ a secure VRF $\implies$ $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$ a secure VRF

Goal: Adversary $\mathcal{A}^{\mathsf{PSPACE}, \mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$ against psuedorandomness of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}$.

Exposed vs. Obscure Queries

Honest evaluation $(y, \pi) \leftarrow \Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$; let τ be the verification trace. Classify each oracle-query vertex of the Eval DAG:

- query to $\mathcal{O}_{uppc}$ $\rightarrow$ default $\mathbf{x}_{\mathsf{v}}^{def}$
- query to $\mathcal{O}_{vrf}$ in τ $\rightarrow$ *exposed*: read from τ
- query to $\mathcal{O}_{vrf}$ not in τ $\rightarrow$ *obscure*

Claim (obscure queries are irrelevant)

Under any well-formed reassignment of responses to the obscure queries, the combiner still outputs the same image y.

Sketch. Else two reassignments yield $\widehat{\mathcal{O}}_{\mathsf{vrf}}, \widetilde{\mathcal{O}}_{\mathsf{vrf}}$ with $\hat{y} \neq \tilde{y}$. The *vrfification lemma* upgrades $\mathcal{O}_{uppc}$ to a secure VRF matching it on the relevant keys — τ is the same in both worlds, breaking unique provability. $\Rightarrow\Leftarrow$

A Query-Free Simulator $\mathsf{EvalSim}_{\mathsf{VRF}}$

Given $(\mathsf{sk}, x, y, \pi, \tau)$, simulate the combiner's Eval without any oracle call: answer $\mathcal{O}_{uppc}$ queries with defaults, exposed queries from τ, and *branch* over all well-formed responses on obscure queries. Return $\top$ iff every branch outputs y.

Properties

- $\mathsf{EvalSim}_{\mathsf{VRF}}$ makes no queries to $\mathcal{O}_{vrf}, \mathcal{O}_{uppc}$.
- $\mathsf{sk} = \mathsf{sk}^* \Rightarrow \mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) = \top$ (previous claim)
- $\Pi_{\mathsf{VRF}}.\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}, x) \neq y \Rightarrow \mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) = \bot$

Inefficient, but decidable in PSPACE.

The Adversary

1. Sample $x_1, \ldots, x_{s(\lambda)+\lambda} \leftarrow_\$ \{0,1\}^\lambda$; obtain (y_i, π_i, τ_i) via $\mathcal{O}_{\mathsf{eval}}$ and $\Pi_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}$.
2. Use PSPACE to compute the filtered set $\mathcal{SK}_{cand} = \bigcap_i \{\mathsf{sk} \in \mathcal{SK} : \mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \top\}$; note $\mathsf{sk}^* \in \mathcal{SK}_{cand}$.
3. Sample $\mathsf{sk}_{cand} \leftarrow_\$ \mathcal{SK}_{cand}$ uniformly.
4. On challenge x^*, predict $y' \leftarrow \Pi_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x^*)$ and guess $b = 0 \iff y' = y^*$.

Advantage. Call sk *unfavorable* if it agrees with sk^* on a random x with probability $< 1/2$. A Markov bound shows unfavorables survive all rounds with expected count $\leq 2^{-\lambda}$, hence

$$\mathbf{Adv}^{\mathsf{prf}}_{\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}}(\lambda, \mathcal{A}) \geq \tfrac{1}{2} - \tfrac{1}{2^{\lambda+1}} - \tfrac{1}{|\mathcal{Y}|}.$$

From $(2; 1)$ to $(n; \lfloor n/2 \rfloor)$

Given a robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner Π_{VRF}, build a robust $(2; 1)$ combiner $\widetilde{\Pi}_{\mathsf{VRF}}$:

- initialize the first $\lfloor n/2 \rfloor$ candidates of Π_{VRF} with $\mathcal{O}_1$
- initialize the remaining $\lceil n/2 \rceil$ candidates with $\mathcal{O}_2$

If either $\mathcal{O}_1$ or $\mathcal{O}_2$ is a secure VRF, at least $\lfloor n/2 \rfloor$ candidates of Π_{VRF} are secure, so $\widetilde{\Pi}_{\mathsf{VRF}}$ is a robust $(2; 1)$ combiner.

Contraposition with the main theorem yields the impossibility of robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiners.

What We Showed

Theorem. Impossibility 1-out-of-2 combiner.

There is **no** robust $(2; 1)$ black-box VRF-combiner.

Corollary. Impossibility $\lfloor n/2 \rfloor$-out-of-n combiner.

There is **no** robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner.

Generalizing to $(n-1)$-out-of-n Combiner

Conjecture. Impossibility $(n-1)$-out-of-n combiner.

There is **no** robust $(n; n-1)$ black-box VRF-combiner.

Where the Proof breaks down

- In the 1-out-of-2 case we rely on the fact that for one candidate we can always guess the responses perfectly, and for the other the obscure queries are mutually irrelevant in the sense that under the correct secret key the image y stays the same. That means that for all tests on the correct secret keys sk^* the output remains consistent through all runs. For any $n > 2$ and $k = n - 1$ we have at least two candidate whose outputs we cannot predict perfectly. While mutual independence holds for

Beyond Impossibility

Let assume *unique provability* for all *n* candidates:

- *Then a parallel combiner for PRF is applicable*

Too strong of an assumption?

- No, unique provability is an information theoretic property and typically proven.

Open Questions

Conjecture: $(n; n-1)$ is also impossible

There exists no robust $(n; n-1)$ black-box VRF-combiner.

That is, it is *not* enough for all-but-one candidates to be secure — any robust black-box VRF-combiner needs strictly more than a constant number of insecure candidates to tolerate.

Why it is hard. Our $(2; 1)$ impossibility proof exploits a specific structural argument about obscure queries and verification traces. Extending it to $n-1$ adversarial candidates would require reasoning about the simultaneous freedom on many oracles, and the current technique does not directly generalize.

Other open directions.

- Closing the gap between $(n; \lfloor n/2 \rfloor)$ (impossible) and $(n; n-1)$ (conjectured impossible).
- Finding the weakest candidate assumptions under which *some* robust black-box VRF-combiner exists.

Summary

Summary

- Introduced Robust $(n; k)$ Black-Box DAG Combiner for more principled analysis
- Established impossibility for robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner
- Conjectured impossibility for robust $(n; n - 1)$ black-box VRF-combiner
- Assuming unique provability for all candidates allows for rboust $(n; 1)$ black-box VRF-combiner